Ondas de choque con OpenMP

Compilar, ejecutar, medir y analizar

Objetivo. Observar la compresión de un cristal de argón y medir cuánto disminuye el tiempo de ejecución al usar varios hilos. Trabaja en parejas; registra el procesador, los núcleos disponibles y los resultados obtenidos.

1. Preparar y compilar

Se requiere Linux, CMake ≥ 3.16, C++17, OpenMP y zlib. Para los gráficos usaremos Python 3 y Matplotlib. En Debian/Ubuntu, si faltan dependencias:

```
sudo apt-get update
sudo apt-get install build-essential cmake zlib1g-dev \
  python3 python3-matplotlib
```

Abre una terminal **en la raíz del repositorio lpmd2**. Mantén esa misma terminal durante la práctica: las variables siguientes se usarán en las tres páginas.

```
repo="$PWD"
ejemplo="$repo/lpmd/examples/shock-wave"
cmake -S "$repo" -B "$repo/build-openmp" \
  -DCMAKE_BUILD_TYPE=Release -DLPMD_ENABLE_OPENMP=ON \
  -DLPMD_ENABLE_TESTS=ON
cmake --build "$repo/build-openmp" --parallel 4
ctest --test-dir "$repo/build-openmp" --output-on-failure
export LPMD_PATH="$repo/build-openmp/lib"
export OMP_DYNAMIC=FALSE OMP_PROC_BIND=close OMP_PLACES=cores
```

La configuración debe indicar `Found OpenMP_CXX` y las pruebas deben pasar. Se trabaja directamente con los ejecutables compilados; no hace falta instalar LPMD en el sistema ni abrir una ventana gráfica.

Dos controles distintos: `--parallel 4` usa hasta cuatro procesos para **compilar**. `OMP_NUM_THREADS=4` solicita cuatro hilos para **simular**. Si dispones de menos núcleos, ajusta los hilos a tu equipo.

2. Verificar con una corrida corta

```
mkdir -p "$repo/runs/clase-shock/smoke"
cd "$repo/runs/clase-shock/smoke"
OMP_NUM_THREADS=4 OMP_DISPLAY_ENV=TRUE \
  "$repo/build-openmp/bin/lpmd" "$ejemplo/smoke.control" \
  > run.log 2>&1
grep -E 'OMP_NUM_THREADS|SIMULATION FINISHED' run.log
```

Comprueba el número de hilos solicitado y `SIMULATION FINISHED`. Se generan `smoke.xyz`, `smoke-monitor.dat` y `smoke-final.dump`: 1152 átomos durante 100 pasos, equivalentes a 50 fs. Esta corrida verifica el programa; todavía no desarrolla el frente de compresión.

La corrida y la comparación de tiempos

3. Simular la compresión

El control `shock.control` crea dos mitades de un cristal FCC frío de Ar. La mitad inferior viaja hacia $+z$ y la superior hacia $-z$: se encuentran cerca del centro y generan frentes de compresión.

Sistema	6912 átomos; celda de $32.4 \times 32.4 \times 259.2$ Å; parámetro de red $a = 5.4$ Å.
Interacciones	Lennard-Jones; $\sigma = 3.41$ Å, $\epsilon = 0.0103408$ eV y corte de 8.5 Å.
Movimiento	Velocidades iniciales de ± 0.004 Å/fs (± 400 m/s); contornos periódicos en las tres direcciones.
Integración	Velocity Verlet; $\Delta t = 0.5$ fs y 4000 pasos.

```
mkdir -p "$repo/runs/clase-shock/choque"
cd "$repo/runs/clase-shock/choque"
OMP_NUM_THREADS=4 \
  "$repo/build-openmp/bin/lpmd" "$ejemplo/shock.control" \
  > run.log 2>&1
grep 'SIMULATION FINISHED' run.log
```

`shock.xyz` contiene 40 configuraciones, guardadas cada 100 pasos: desde 50 fs hasta 2000 fs. `shock-monitor.dat` contiene, en orden, paso, energía potencial, cinética y total (eV), temperatura (K), presión y esfuerzo configuracional s_{zz} (MPa).

Tiempo físico: 4000×0.5 fs $= 2$ ps. **Tiempo de reloj:** lo que tarda el computador. Más hilos pueden reducir el segundo sin cambiar el primero. Conserva el paso temporal para esta comparación.

4. Medir con 1, 2 y 4 hilos

El siguiente comando usa la misma geometría y condición inicial, pero solo 200 pasos; repite cada caso tres veces y muestra la mediana del tiempo de reloj.

```
cd "$repo/runs/clase-shock"
python3 "$ejemplo/benchmark.py" "$repo/build-openmp/bin/lpmd" \
  --plugin-path "$repo/build-openmp/lib" \
  --threads 1 2 4 --steps 200 --repeats 3 --no-trajectory \
  --output tiempos
```

`--no-trajectory` omite XYZ en todos los casos, pero conserva monitor y estado final: se mide el proceso completo con menos escritura. Ejecuta las mediciones una tras otra, sin otras simulaciones, y usa solo los núcleos asignados. Los datos quedan en `tiempos/times.csv`; el control original no se modifica.

Completa la tabla usando la mediana $\widetilde{t}_p$ de las tres repeticiones:

$$S_p = \frac{\widetilde{t}_1}{\widetilde{t}_p}, \qquad E_p = 100\,\frac{S_p}{p} \quad \text{(eficiencia en \%)}.$$

Hilos p	Mediana $\widetilde{t}_p$ (s)	Aceleración S_p	Eficiencia E_p (%)
1	_____	1.00	100
2	_____	_____	_____
4	_____	_____	_____

Gráficos y análisis de los resultados

5. Construir los perfiles y graficar

Vuelve a la carpeta de la corrida completa. Estos comandos generan el CSV de perfiles y dos figuras PNG, sin necesidad de abrir una interfaz gráfica.

```
cd "$repo/runs/clase-shock/choque"
python3 "$ejemplo/profiles.py" shock.xyz --output perfiles.csv
python3 "$ejemplo/plot_results.py" perfiles.csv \
  --times ../tiempos/times.csv
```

- `shock-profiles.png`: densidad relativa n/n_0 y velocidad media longitudinal $\langle v_z \rangle$ frente a z, a 0.5, 1 y 2 ps.
- `shock-speedup.png`: aceleración medida frente a hilos solicitados y recta ideal $S_p = p$.

Los perfiles agrupan los átomos en 24 franjas de 10.8 Å. La densidad de referencia y la conversión temporal son:

$$n_0 = \frac{4}{a^3} = \frac{4}{(5.4\ \text{Å})^3} \simeq 0.02540\ \text{Å}^{-3}, \qquad 1000\ \text{fs} = 1\ \text{ps}.$$

En el CSV, `time_fs` está en fs, `z_angstrom` en Å, `number_density_A^-3` en Å^{-3} y `mean_vz_m_s` en m/s. La primera configuración corresponde a 50 fs, no a la condición inicial. Una franja sin átomos tiene densidad cero y velocidad media vacía.

Para repetir: usa un directorio de trabajo nuevo. El simulador reemplaza sus salidas; los scripts protegen los CSV, directorios de medición y PNG existentes. Para guardar otras figuras, añade `--output-dir figuras-2` al comando de gráficos.

6. Responder y discutir

1. **Compresión.** Identifica la zona central donde $n/n_0 > 1$. ¿Cómo cambia su extensión entre 0.5 y 2 ps? Compara la velocidad media dentro de esa zona con las velocidades de las regiones aún sin comprimir.
2. **Propagación.** Sigue el borde derecho de la zona comprimida entre 1 y 2 ps. Usa como criterio aproximado $n/n_0 > 1.1$ en las franjas contiguas al centro y estima $v_f \simeq \Delta z/\Delta t$. Recuerda: $1\ \text{Å/ps} = 100\ \text{m/s}$. ¿Cuánto limita la estimación el ancho de una franja?
3. **Rendimiento.** ¿Cuánto se redujo el tiempo con cuatro hilos? Calcula S_4 y E_4. Explica dos razones por las que la aceleración puede ser menor que cuatro: trabajo secuencial, sincronización, memoria o carga del equipo. Si $S_p < 1$, ¿qué significa?
4. **Interpretación física.** El cristal parte sin velocidades térmicas aleatorias, pero su temperatura global no es cero. ¿Qué aporta el movimiento colectivo? ¿Por qué la temperatura del monitor no basta para medir el calentamiento local por el choque?

Alcance del modelo. Es una compresión transitoria, sin termostato. La unión periódica de los extremos genera una descarga: distingue ese efecto del frente central. El cristal frío presenta oscilaciones entre planos atómicos. Además, s_{zz} solo incluye interacciones y la temperatura global incluye flujo; el corte del potencial puede introducir cambios en la energía total.

Entrega breve: las dos figuras, la tabla de tiempos completada y respuestas de pocas líneas. Indica procesador, núcleos disponibles, hilos, pasos y repeticiones para que otra pareja pueda reproducir la medición.